

Lecture 1 Cryptocurrency

1. Cypherpunk Philosophy & Monetary Foundations

(1) Cypherpunk Philosophy: Anti-centralization, replacing trust with protocols, and converting "trust" into "algorithmic guarantees".

--Technological impact: PGP (Secure Email), Tor (Anonymous Browsing), Bitcoin (Decentralized Digital Currency).

(2) Two Prerequisites of a Monetary System: 1. Scarcity: Currency must be minted legitimately. Scarcity is a necessary but not sufficient condition.

2. Ownership: Currency must belong to the legitimate holder.

--The Core Point: The system must prevent the Double Spending problem.

2. Double Spending Defense & Ledger Systems

(1) Mechanisms to Prevent Double Spending

- ✓ Multiple complete copies: Full nodes (e.g., miners, major wallet providers) store the entire blockchain ledger.
- ✓ Public visibility: All transactions are publicly broadcasted to the network.
- ✓ Immutable history: Relies on Merkle Roots and Hash Pointers for decentralized synchronization.
- ✓ Quasi real-time synchronization: Uses Nakamoto Consensus (Longest Chain Rule) to allow distributed nodes to "eventually reach agreement".

(2) Bitcoin's core innovation is the introduction of Triple-Entry Bookkeeping, which adds a "public cryptographic receipt" to traditional debits and credits.

- ✓ Traditional Banks [Double-Entry], validation Method [Debits & credits must balance], Trust in a centralized institution, Failure Risk [Single point of failure]
- ✓ Bitcoin [Triple-Entry], Validation Method [Cryptographic signature verification], Trust in Cryptography + Distributed consensus, Failure Risk [Distributed redundancy]

(3) UTXO (Unspent Transaction Output) Model

- ✓ Pros: Efficiently prevents double

spending, supports parallel verification, and offers potential for privacy (if a new address is used for every transaction).

- ✓ Cons: Calculating a total balance requires summing up all relevant UTXOs; it lacks a shared state, which is disadvantageous for implementing complex smart contracts.

3. Core Cryptographic Components

(1) Hash Functions: it maps data of arbitrary size to a fixed-size output (e.g., 256 bits). It exhibits the Avalanche effect, is strictly one-way, and does not provide confidentiality.

- ✓ Preimage Resistance: Prevents reverse engineering. (Given h finding m such that $\text{Hash}(m) = h$ is computationally infeasible.), Application [Mining]
- ✓ Second Preimage Resistance: Prevents tampering with commitments. (Given m_1 , finding m_2 such that $\text{Hash}(m_1) = \text{Hash}(m_2)$ is computationally infeasible.), [Block Linking]
- ✓ Preimage Resistance: Prevents reverse engineering. (Finding any two $m_1 \neq m_2$ such that $\text{Hash}(m_1) = \text{Hash}(m_2)$ is computationally infeasible.), Application [Transaction IDs]

(2) Digital Signatures
Provides authentication and authorization, but does not provide privacy or deniability.

- ✓ ECDSA (Elliptic Curve Digital Signature Algorithm): The traditional Bitcoin signature algorithm.
- ✓ Schnorr Signatures: Introduced via the Taproot upgrade. Fixed size of 64 bytes, supports batch verification and linear aggregation (making multi-sig transactions look identical to single-sig ones), significantly improving both privacy and efficiency.

(3) Merkle Trees

- ✓ Architecture: Built bottom-up. Adjacent transaction hashes are concatenated and hashed again, eventually generating a single Merkle Root stored in the block

header. Modifying any underlying transaction alters the Root completely.

- ✓ Advantage: Provides $O(\log n)$ complexity for transaction verification.
- ✓ SPV (Simplified Payment Verification / Light Clients): Enables light clients to efficiently verify "Proof of Inclusion" (whether a transaction is included in a specific block).
- ✓ SPV Limitations: SPV cannot verify if a transaction is a double spend, nor can it verify if a block violates consensus rules.
- (4) Mining & Longest Chain Rule
- ✓ Mining: Finding a Nonce through brute force such that $\text{Hash}(\text{BlockHeader}) < \text{Target}$.
- ✓ Longest Chain Rule (Nakamoto Consensus): During a temporary network fork, miners will choose to extend the chain with the greatest accumulated Proof of Work.
- ✓ Orphan Blocks: Blocks that are isolated and not included in the longest chain will be discarded. Their transactions are returned to the Mempool to await repacking.
- 4. System Security & Bitcoin Script
- (1) Threat Model & Defense
- ✓ Sybil Attack: An attacker subverts the reputation system by creating a large number of pseudonymous/fake identities to out-vote honest nodes.
- ✓ Defense (PoW): Proof of Work binds "voting power" directly to "physical computational power," rendering the creation of logic-based fake identities useless, thus achieving Sybil resistance.
- (2) Bitcoin Script: A stack-based programming language used to lock and unlock UTXOs.
- ✓ Characteristics: It is Non-Turing complete. Looping constructs are intentionally omitted to prevent infinite loops and Denial of Service (DoS) attacks, ensuring that transaction verification is deterministic and secure.

Lecture 2: Consensus Mechanisms

1. Consensus Properties (共识三性质)

Agreement: all correct nodes $\rightarrow$ same value

Validity: decided value $\in$ proposal of correct node

Termination: all honest nodes eventually decide (prevent DoS)

premise: Digital Signature + PKI (authentication)

2. (The Byzantine Generals Problem)

CFT (Crash Fault Tolerant): node crash only

BFT (Byzantine Fault Tolerant): arbitrary / malicious behavior

3. CAP Theorem

Consistency / Availability / Partition Tolerance

4. BB vs SMR

(1) Byz. Broadcast (BB)

one-shot consensus (单次共识)

Leader propose value

Synchronous: Dolev-Strong ($f < n$):

签名链

Asynchronous: FLP Theorem

(deterministic consensus impossible)

(2) State Machine Replication

(SMR)

continuous consensus (持续共识)

agree on ordered log

Participants: (1) Replicas (run

consensus) (2) Clients (query

state)

5. PBFT (Practical BFT)

场景: permissioned / consortium

blockchain

cons:

Communication complexity: $O(n^2)$,

not scalable;

vulnerable to Sybil attack in

Permissionless network

6. Nakamoto Consensus

k-confirmation / k-deep rule

wait k blocks

attacker power $\beta < 0.5, k \rightarrow \infty \rightarrow$ attack

probability $\downarrow$ exponentially

Security condition

$\beta < 1/2$ ($1/\Delta/T$)

Δ : network delay

T : block interval

👉 $\Delta \uparrow$ 或 $T \downarrow \rightarrow$ security margin $\downarrow$

7. Modern BFT

(1) Tendermint

$O(n^2)$ communication complexity $\rightarrow$

Achieve instant finality

Use Slashing punishment mechanism

(2) HotStuff

$O(n)$ communication (linear)

signature aggregation (聚合签名)

3-chain commit rule

8. DAG-based Consensus

multi-parent blocks $\rightarrow$ DAG

Decouple data availability from

consensus (ordering)

Pros:

high throughput

low latency

protocol:

(1) Avalanche: leaderless + repeated

random sampling (Snowball

protocol) on k random nodes until

convergence

(2) Narwhal / Tusk & Mysticeti

Mysticeti: 390ms latency / 100K +

TPS

9. MEV (Maximal Extractable Value)

定义:

profit from transaction ordering

attack:

Sandwich attack

Arbitrage

Liquidation

defense:

Encrypted mempool

Fair ordering protocol

10. Fork

Soft Fork

backward compatible

new rule is stricter rules

eg: SegWit, Taproot

Hard Fork

not compatible,

ecosystem fails to reach an agreement

lead to permanent chain split

eg: Bitcoin Cash

Lecture 3: Ethereum

1. UTXO

UTXO is a specific transaction output that has not been spent yet, identified by (txid, output index), and it carries a value plus a spending condition (e.g., a signature/public-key check).

(2) Validation Rules

- ✓ ScriptSig + ScriptPK $\rightarrow$ TRUE
- ✓ TxID/index exist in current UTXO set.
- ✓ $\text{Sum}(\text{Inputs}) \geq \text{Sum}(\text{Outputs})$

. (Pass -> Remove spent UTXO & New outputs -> added to UTXO set).

2. Bitcoin Script & Security

2.1 P2PKH (Pay-to-PubKey-Hash)

(1) Structure

- ✓ ScriptSig: <sig> <pubkey>
- ✓ ScriptPK: DUP HASH160 <PubKeyHash> EQUALVERIFY CHECKSIG

(2) Exec Flow: Sig -> Pubkey -> DUP P -> 计算 HASH160 -> Target Hash -> EQUALVERIFY -> CHECKSIG -> TRUE.

(3) Security Properties:

- ✓ Privacy: pubkey hidden before spending
- ✓ Quantum resistance: hash protects pubkey
- ✓ Anti-tampering: cannot change address (breaks signature)

2.2 P2SH (Pay-to-Script-Hash)

- ✓ Pay to redeem_script hash.
- ✓ Reveal script when spending
- ✓ Used for Multisig (多签, e.g., 2-of-3 Multisig).

3. Ethereum Fundamentals

(1) Vision: General DApp computer, Supports DApps + Smart Contracts.

(2) Architecture

3 Layers: Consensus (PoS), Compute (EVM), State (stores state changes).

(3) Account Model:

EOA (外部账户): Controlled by private key

- ✓ Addr = H(pk). No code/storage.
- ✓ Nonce = tx count.

Contract (合约账户): Code controlled.

- ✓ Has StorageRoot & CodeHash.
- ✓ Addr = H(CreatorAddr, CreatorNonce).
- ✓ Nonce = 创建合约数.

(4) Transactions (交易类型):

- ✓ Fields: Sender, To, Amount, Data, Gas, Nonce, Sig
- ✓ Types: Transfer: To = EOA, Data = Empty. Creation (部署): To = 0x0, Data = Bytecode. Invocation (调用): To = Contract, Data = Calldata.

✓ Rule:

(1) Determinism (Not allowed: random/ external API/float) (2) Atomicity (All succeed OR all revert).

(5) Gas Mechanism:

- ✓ Prevents infinite loops (
- ✓ compensate the validator for the computing resources

Fee = gasPrice * gasUsed.

Over limit: State revert, BUT gas NOT refunded

EIP-1559: Fee = Base Fee (burned) + Priority Fee (validator tip).

4. Smart Contracts

(1) Solidity: High-level language, Compiles to EVM bytecode
(2) EVM: Stack machine (Max 1024), JUMP (for loops)

(3) Global Variables

msg.sender: Direct caller

msg.value: Wei sent.

tx.origin: Original EOA. DON'T USE for auth

(4) Error Handling

require() -> input check (refund gas)

assert() -> bug check (consume gas)

(5) fallback:

No selector match -> fallback().

Pure ETH transfer -> receive().

(6) Data Locations

Storage ~~非易失~~: permanent

Memory ~~易失~~: Temporary

Calldata ~~非~~: only reads in parameters

Stack ~~非~~: basic type

(7) Gas Opt (gas 优化)

- ✓ compute in memory
- ✓ minimize storage writes
- ✓ Constant use Calldata
- ✓ Transfer use .call(value: amount) (**)

4. Security

(1) Reentrancy Attack

- ✓ How: EVM is single-thread. External call -> attacker fallback() -> re-enter function. state not updated yet -> repeated withdrawal

- ✓ Defense (CEI Pattern): 1. Checks (require verify) 2. Effects (update balance FIRST) 3. Interactions (External call).

(2) Replay Attack

- ✓ How: Intercept a signed and

successful transaction -> Resend signed tx.

- ✓ Defense: Nonce. strictly increasing, must match exactly After execution discarded Nonce.

5. Token & DeFi

(1) ERC20 token

a blockchain-based fungible, asset with functionalities similar to Ethereum's native cryptocurrency ETH, but operates within the Ethereum platform, conforming to a specific set of standards.

- ✓ Functionalities: totalSupply, balanceOf, transfer, transferFrom, approve, allowance

(2) DeFi / Stablecoins:

- ✓ defi: Protocols can call each other without permission [Flash Loan]

(3) Custodial (USDC/USDT): 1:1 USD fiat. Centralized (Freeze risk).

- ✓ Crypto-backed (DAI): Smart contract. Over-collateralized (超额抵押). Liquidation (清算): If collateral ratio < threshold -> system sells collateral

Lecture 4 Proof of Stake

1. PoS Key Properties

- ✓ Dynamic Availability: System continues even if participation changes.
- ✓ Finality: Once finalized -> irreversible.
- ✓ Accountable Safety: Misbehavior -> detectable + punishable (slashing).
- ✓ Energy Efficiency

2. PBFT-style Safety

Attack: Byzantine Double-Signing (双重签名)

- ✓ Malicious validator signs conflicting blocks in same epoch

Slashing Process: Observer detects evidence, Submit to On-chain Contract, Contract verifies proof, Execute Burn Stake

3. Slashing Conditions

- (1) Double Voting Same epoch vote for two different targets (A & B)
- (2) Surround Voting One vote interval contains another

(source₁ -> target₁) surrounds (source₂

-> target₂)

(3) Safety Conclusion

- ✓ These are the ONLY ways to break Gasper BFT safety
- ✓ Signed -> non-repudiable
- ✓ Detected -> immediate slashing

4. Last Reveal Problem

Last proposer can influence randomness by withholding block reward

5. Proposer-Builder Separation (PBS)

Roles:

- ✓ Builder: Build blocks + extract MEV
- ✓ Relay: Collect bids from Builder + verify blocks + send highest bids to validator
- ✓ Validator / Proposer: Choose highest bid, sign and propose

Impact:

Separate block construction & proposing MEV shared via auction

Lecture 5: DeFi Fundamentals

1. Core Changes of DeFi

- ✓ Access: From bank permission -> Internet access (permissionless)
- ✓ Custody (资金控制): From bank holds funds -> user controls private key
- ✓ Operating Time: From business hours -> 24/7/365
- ✓ Transparency: From hidden fees -> open-source & auditable code
- ✓ Profit Margin: From 3-5% intermediaries -> 0.1-0.5% protocol fee

2. Yield Sources (收益来源)

Trading Fees; Loan Interest

Staking Rewards; Token Incentives

3. Risks in DeFi

(1) Risk Premium (风险溢价)

Yield > 4% => compensating for risks: Smart Contract Risk; Protocol Risk; Liquidity Risk; Regulatory Risk

(2) Rug Pull Warning Signs 跑路信号

Anonymous Team; No Audit;

No Timelock; Guaranteed High APY;

Unfair Token Unlock

Lecture 6: privacy

1. Bitcoin Anonymity & Heuristics

- (1) Pseudo-anonymity: Only real-

world identity is hidden. Addresses are public

(2) Breaking Anonymity

- ✓ Common Input Ownership (CIO): Multiple inputs in one transaction -> controlled by the same entity.

- Exceptions: CoinJoin (multiple users) & PayJoin (sender + receiver).

- ✓ Change Address Detection: Identifying (unused fund) the return address.

- Clues: Integer payment; New address; same address type; matching time zone/spending pattern

2. CoinJoin

Properties:

- ✓ Atomic: All or nothing.
- ✓ Equal Amounts: Prevents amount matching
- ✓ Randomized Output: Prevents position matching.

3. PayJoin (Pay-to-Endpoint)

A CoinJoin disguised as a normal TX. Both sender and receiver provide inputs.

4. Pedersen Commitments

Cryptographic commitment with additive homomorphism

(1) Purpose: Verify sum(inputs) == sum(outputs) without revealing amounts

(2): Range Proofs

- ✓ Prevent negative value attack
- ✓ Prove: value ∈ [0, 2⁶⁴) without revealing actual value

5. Stealth Address (One-time Address)

(1) Key Value

Prevents Address Reuse; hides total income/spending from observers.

(2) based on ECDH

- ✓ Step 1: Receiver publishes scan key: S = s * G, Spend key: B = b * G
- ✓ Step 2: Sender picks random r, computes R = r * G, Shared key: k = H(rS) = H(rsc), generate One-time address: P = k * G + B, sends funds and publishes R
- ✓ Receiver scans R, Compute k = H(s * R), derives Recover P = k * G + B

6. Ring Signatures

(1) What verifier knows?

Signer E group (ring)

(2) What v ~ Don't know?

Which member signed

(3) Key Image

- ✓ unique tag Prevents double spending
- ✓ Same key image → rejected tx
- ✓ Identity still hidden

7. Monero Privacy Stack

(1) Technology Stack

- ✓ Ring Signatures → hide sender
- ✓ Stealth Address → hide receiver
- ✓ RingCT/Bulletproofs → hide amount

(2) Privacy Paradox (Opt-in Problem)

- ✓ Zcash: <10% use privacy → users stand out
- ✓ Monero: 100% privacy enforced → large anonymity set

8. Tornado Cash

- ✓ First Sanctioned by OFAC to smart contracts

Its Ethereum mixer using ZK proofs
→ Debate is code == speech?

9. Privacy Pools (Vitalik)

Users withdraw with ZK proof → Prove funds E clean set → Exclude us are hackers/criminals with protecting privacy

Lecture 7: Zero-Knowledge Proofs

1. ZKP Fundamentals

(1) ZKP (Zero-Knowledge Proof):
Prover convinces Verifier they know secret info (Witness) related to a public statement without revealing the secret itself.

(2) 3 Guarantees

- ✓ Completeness → honest prover convinces verifier
- ✓ Soundness → Cheating prover cannot convince verifier.
- ✓ Zero-Knowledge → no information leakage

2. SNARKs (ZK-SNARKs)

(1) Definition

- ✓ S (Succinct) → tiny proof (~192 bytes), fast verification (~ms)
- ✓ N (Non-interactive) → single message
- ✓ AR (Argument) → computational soundness
- ✓ K (Knowledge) → prover must know witness

(2) Main Systems

- ✓ Groth16: Smallest proof (~192B), fastest verification, but Per-circuit trusted setup required
- ✓ PLONK: Universal setup (one ceremony for many circuits), Easier smart contract upgrades

(3) Trusted Setup

Generates secret parameter: $\tau, \tau_2, \tau_3, \dots$,
Used in polynomial commitments (QAP)

(4) Toxic Waste: The generated random parameter τ

Secret τ must be destroyed

If leaked → attacker can forge proofs
Based on Schwartz-Zippel Lemma
attack → forge polynomials that evaluate correctly at τ but are wrong everywhere else

(5) MPC Ceremonies

Multiple people participate and contribute randomness

- ✓ Security rule:

≥ 1 honest participant → safe
Honest party destroys secret → τ unknown

3. STARKs (ZK-STARKs)

Properties

- ✓ No trusted setup (transparent)
- ✓ Quantum-resistant (hash-based)
- ✓ Large proofs (50-200 KB)
- ✓ Slower verification (10-50 ms)

4. Bulletproofs

Properties

- ✓ No trusted setup
- ✓ Proof size: $O(\log n)$
- ✓ Verification time: $O(n)$ (slow)
- ✓ Not quantum-resistant

Lecture 8: Blockchain Scaling

1. Replication Bottleneck (Why blockchain is slow)

- ✓ Every full node re-executes every transaction
- ✓ More nodes $\Rightarrow$ more security, NOT more speed
- ✓ Throughput depends on slowest node

2. Desirable L2 Properties

- ✓ Public Transparency → Anyone can compute the same state as the operator
- ✓ Self-custody → Only you can authorize changes to your

balance

- ✓ Censorship Resistance → You can force your transaction to be executed

3. Payment Channels

(1) Lifecycle

Open (deposit funds on-chain) →

Use (Sign state updates off-chain) →

Close (Submit final state on-chain)

(2) One-way Channel (Spilman)

Uses Replace-by-Incentive
Receiver keeps latest (largest amount) transaction

(3) Two-way (Lightning)

Uses Replace-by-Revocation

- ✓ Mechanism: Uses 2-of-2 multisig & asymmetric commitment txs (Broadcaster waits via timelock; Counterparty gets funds immediately).

- ✓ Revocation: Must exchange old revocation keys on each state update.

- ✓ Penalty: If malicious party broadcasts an old state → Honest party uses old key to fire a Justice Tx and take ALL funds.

4. Rollup

(1) Core Idea:

Move computation off-chain (amortize L1 gas for hundreds of L2 txs), keep data & validation on L1.

(2) Structure

- ✓ L1: Validating Bridge → holds funds + verifies
- ✓ (Off-chain) Sequencer → collects & orders txs, sends batch to L1

- ✓ (Off-chain) Executor → executes txs, computes new state, generates proof

(3) Agents

- ✓ Honest User → send TX, seek security
- ✓ Sequencer → Collect & ordering tx + checkpoint submission to L1
- ✓ Executor → execute tx, computes new state, proves correctness.

(4) Workflow

Deposit (L1 Validating Bridge) →
Transfer (Off-chain) → Checkpoint
(Sequencer sends batch + new root to L1) → Execute (Submit proof of correct execution to Bridge).

5. Fraud Proof Bisection Protocol (欺诈证明二分法)

(1) Why need it?

L1 gas is extremely expensive; cannot re-execute a whole batch of txs (e.g., 500 txs) on L1 to verify.

(2) How it works?

- ✓ Interactive protocol:

$O(\log_2 N)$ rounds

- ✓ Steps:

1. Split execution into halves

2. Compare state hashes

3. Narrow dispute repeatedly

4. Find exact 1-step

disagreement → L1 only executes ONE step (Huge gas saving)

6. Data availability

(1) Problem

If Sequencer only posts state hashes but hides tx data, users cannot generate fraud proofs for bad states → Funds get frozen.

(2) Solution

Post on L1: Must publish L2 tx data to L1 to strictly inherit its security.

考试可能出的题目:

1. UTXO: How does value move i/o?

Each input points to a previous output via (txid, index). Each output creates a new UTXO locking value to a recipient. $\text{Sum}(\text{Inputs}) \geq \text{Sum}(\text{Outputs})$.

2. hash pointer?

"reference + hash." pointer/reference locates data, hash verifies integrity; hash alone cannot retrieve data

3. PBFT VS NC

- ✓ Fault assumption: PBFT faults up to $<1/3$ of validators; NC guarantees are probabilistic
- ✓ Finality: PBFT has deterministic finality; Nakamoto has probabilistic finality

3. 构建 reliable broadcast

- ✓ Define Committee: Fix a public committee of size n that includes the sender S

- ✓ Execute Protocol: Only the committee runs protocol once. The sender's bit b is the input.

- ✓ Relay Step: Whenever a committee member i 属于 C , it sends an authenticated message

(RELAY, v) to all m parties.

- ✓ Output Rule: Any party outputs v as soon as it receives (RELAY, v) from $f+1$ distinct committee members.

4. Merkle tree Advantages

- ✓ A block can commit to a large dataset with a single digest.
- ✓ To prove inclusion of a single item (transaction, receipt, state leaf), one supplies only $O(\log n)$ hashes,

5. How PoS validates transactions?

1. Stake (lock collateral) 2. Select a proposer. 3. Other validators verify and vote (attest) 4. Finalize, reward, and punish

6. pos vs pow?

Pros: 1. energy efficiency: it avoids energy-intensive mining 2. PoS has a Slashing mechanism
Cons: if a small number of entities control a large fraction of stake, they may gain outsized influence.

7. Account vs UTXO (为何 ETH 放弃 UTXO?)

- ✓ State Management: supports persistent state
- ✓ Programmability: enables Turing-complete for DApps
- ✓ Performance: Account = Sequential execution, smaller state size, easier for complex DApps

8. Procedures of DEX, use Uniswap explain the solution of AMM?

Uniswap uses the constant product formula: $x * y = k$

Where:

x = quantity of token A in the pool
 y = quantity of token B in the pool
 k = constant product value

Procedures:

- ✓ Liquidity providers deposit/withdraw token pairs $\Rightarrow$ Smart contracts automatically calculate prices
- ✓ Traders swap directly with the pool $\Rightarrow$ Smart contracts automatically calculate prices

9. Incentives and potential losses for LPs in DEXs?

Incentives: 1. Trading Fees 2. Yield Farming Rewards

Loss: Impermanent Loss

10. As Lps, where loss funds go?

Specifically, who gained x USD in this process?

arbitrageurs stand to profit.

Arbitrageurs capitalize on temporary price discrepancies between different markets or within a market before it reaches equilibrium.

11. tx_1 's $maxPriorityFee < tx_2$'s, Will tx_2 be executed before tx_1 ?

No,

- ✓ The producer's ultimate goal = maximize the total profit from a block, includes both tips and any MEV, executing tx_1 before tx_2 may create a more profitable MEV opportunity that outweighs the extra tip offered by tx_2
- ✓ The current block producer might receive tx_1 significantly earlier than tx_2 . If they are building a block and tx_1 is available in their mempool but tx_2 has not yet arrived

12. Why need Business Privacy?

- ✓ Supply Chain: Prevent competitors from knowing supplier pricing.
- ✓ Payroll: Prevent public employee salary exposure.
- ✓ M&A Activity: Prevent leaked acquisition strategies.
- ✓ Fund Operations: Prevent front-running by observers.

13. Network-level De-anonymization attack and defense (No Chain Analysis)

(1) IP Tracking: First broadcaster ≈ sender;

Defense: Dandelion++

(2) ISP Monitoring: ISP sees all connections

Defense: Tor / I2P

(3) Spy Nodes: Many nodes → triangulate source

Defense: VPN / Tor

14. Relationships among ZKP, NIZK, and ZK-SNARK?

$ZK-SNARK \subset NIZK \subset ZKP$.

$ZKP \rightarrow$ general concept

$NIZK \rightarrow$ non-interactive

$ZK-SNARKS$ add succinctness and use computational soundness

(Argument).

15. How to Transform Interactive →

Non-interactive

Use the Fiat-Shamir Heuristic. Replace the verifier's random challenge with a Hash function applied to the Prover's Commitment. The Hash acts as a "Virtual Verifier".

15. Optimistic vs. ZK Rollups (Trade-offs 对比)

(1) Proof: ZK is pessimistic (needs cryptographic SNARK); Optimistic defaults to valid.

(2) Finality/Withdrawal: ZK is fast (instant after verification); Optimistic is slow (7-day dispute period).

(3) Compute & EVM: ZK is computationally heavy (GPU) & hard to EVM-compat (needs zkEVM); Optimistic is light & easy for EVM.

16. Calldata vs. Blobs (EIP-4844)

Calldata: Permanent L1 storage → Very Expensive.

Blobs: Ephemeral data space (discarded after ~18 days) → Massively lowers L2 TX fees.

17. Censorship Resistance (如果 Sequencer 拒绝打包你的提取请求怎么办?)

(1) On-Chain Exodus: Submit withdraw to L1 with deadline. If ignored, freeze L2 and allow direct L1 extraction.

(2) Forced Inclusion: Submit TX directly to L1 Bridge. Executor must process it in order, or bridge rejects new checkpoints.